

TheKnife

Manuale Tecnico

Laboratorio Interdisciplinare B - a.a. 2025/2026
Universita' degli Studi dell'Insubria

Autori:

Rabuffetti Riccardo (756625, VA)

Gorla Davide (756140, VA)

Scarselli Francesco (756661, VA)

1. Introduzione

TheKnife e' una piattaforma client/server che consente di cercare ristoranti nel mondo, selezionandoli per luogo, tipologia di cucina, fascia di prezzo e servizi offerti (delivery, prenotazione online), e di gestire recensioni degli utenti. Il progetto e' stato realizzato secondo le specifiche del corso di Laboratorio Interdisciplinare B, con un modulo **serverTK** che eroga i servizi appoggiandosi a un database relazionale PostgreSQL, e un modulo **clientTK** con interfaccia grafica JavaFX che si connette al server via socket TCP.

Questo documento descrive le scelte architetturali, i design pattern adottati, la progettazione del database e la struttura del codice sorgente. Per le istruzioni di compilazione ed esecuzione si rimanda al file README.txt nella radice del repository; per la descrizione delle funzionalita' dal punto di vista dell'utente si rimanda al Manuale Utente.

2. Architettura del sistema

Il sistema segue un'architettura client/server a due livelli. Il server (**ServerTK**) apre una *ServerSocket* e resta in attesa di connessioni; per ogni connessione accettata viene avviato un nuovo thread (**GestoreClient**), cosi' da poter servire piu' utenti connessi in parallelo da postazioni diverse, come richiesto dalle specifiche. Ogni GestoreClient mantiene lo stato di sessione (l'utente autenticato sulla connessione, se presente) come campo di istanza: essendo un oggetto per thread/connessione, non e' necessaria alcuna sincronizzazione esplicita su questo stato.

Il client (**ClientTK**, applicazione JavaFX) all'avvio chiede l'indirizzo del server, apre una connessione persistente e la utilizza per tutta la sessione applicativa. Le chiamate di rete sono eseguite su thread separati dal thread grafico di JavaFX (classe **EseguiAsync**), riportando poi il risultato sul thread JavaFX tramite *Platform.runLater*, in modo da non bloccare mai l'interfaccia in attesa della risposta del server.

La comunicazione client/server avviene scambiando oggetti Java serializzati (*ObjectOutputStream/ObjectInputStream*) su un'unica connessione socket per sessione: il client invia un oggetto **Richiesta** (operazione + parametri) e riceve un oggetto **Risposta** (esito, eventuale messaggio d'errore, eventuale dato). Questo protocollo e' definito nel package `theknife.common.protocol`, condiviso da client e server, cosi' che entrambi i moduli si affidino alle stesse classi per la serializzazione.

Diagramma architetturale di riferimento: [doc/UML_Architettura_TheKnife.svg](#).

2.1 Perche' socket TCP con oggetti serializzati, e non un protocollo testuale/HTTP

Si e' scelto un protocollo binario basato su serializzazione Java (anziche', ad esempio, JSON su HTTP) perche' semplifica la trasmissione diretta degli oggetti del modello di dominio (Utente, Ristorante, Recensione, ecc.) senza dover scrivere/mantenere una libreria di mappatura oggetto-testo, ed e' pienamente adeguato a un contesto di laboratorio con client e server entrambi Java. Lo svantaggio principale, un accoppiamento piu' stretto tra le versioni di client e server rispetto a un formato testuale, e' accettabile dato che client e server sono sviluppati e distribuiti insieme.

3. Design pattern utilizzati

- **Facade:** ServiziTKImpl (server) offre un'unica interfaccia coerente (ServiziTK) sopra i quattro DAO sottostanti, nascondendo ai chiamanti i dettagli di come le singole operazioni si traducono in query SQL su piu' tabelle.
- **Proxy remoto:** ServiziTKProxy (client) implementa la stessa interfaccia ServiziTK, ma invece di eseguire la logica localmente inoltra ogni chiamata al server tramite la connessione socket. I controller JavaFX invocano i servizi come se fossero locali, ignari della comunicazione di rete sottostante.
- **DAO (Data Access Object):** UtenteDAO, RistoranteDAO, RecensioneDAO, PreferitoDAO isolano il resto del codice dai dettagli JDBC/SQL, esponendo metodi con nomi ed input/output espressi nei termini del dominio applicativo.
- **MVC:** nel client, le viste sono definite in FXML (src/main/resources/fxml), i controller (theknife.client.controller) gestiscono l'interazione e invocano i servizi, il modello (theknife.common.model) e' costituito da semplici classi Java prive di qualunque dipendenza da JavaFX.
- **Polimorfismo/Template per il ruolo utente:** Utente e' una classe astratta con il metodo astratto getRuolo(); Cliente e Ristoreatore lo implementano restituendo un valore fisso. Il ruolo e' quindi espresso dal tipo concreto dell'oggetto (specializzazione totale ed esclusiva), non da un campo modificabile controllato manualmente.
- **Command (semplificato):** Richiesta incapsula un'operazione (Operazione, enum) e i suoi parametri; GestoreClient.elabora() la interpreta e la esegue tramite uno *switch* sull'operazione, senza conoscere i dettagli del chiamante.

Diagramma UML del dominio (classi, ereditarieta', associazioni): **doc/UML_Dominio_TheKnife.svg**.

4. Progettazione del database

Lo schema concettuale ER e' riportato in **doc/ER_TheKnife.svg** ed e' stato tradotto nello schema relazionale implementato in **src/main/resources/sql/schema.sql**, eseguito su PostgreSQL. Le scelte principali:

- **Specializzazione Utente/Cliente/Ristoreatore:** tradotta con un'unica tabella utenti e un attributo discriminante ruolo (tipo enumerativo Postgres ruolo_utente), anziche' con tabelle separate per sottoclasse: Cliente e Ristoreatore condividono esattamente lo stesso insieme di attributi (la differenza e' solo nel comportamento applicativo), per cui tabelle separate avrebbero richiesto join superflui senza alcun beneficio di normalizzazione.
- **Vincolo UNIQUE(email, ruolo)** sulla tabella utenti, anziche' UNIQUE sulla sola email: la stessa email puo' cosi' corrispondere a due account distinti, uno Cliente e uno Ristoreatore (due "profili" della stessa persona), ma non a due account dello stesso ruolo. Il ruolo scelto in fase di login (ServiziTK.login(email, password, ruolo)) e' quindi parte della chiave di ricerca delle credenziali, non solo un filtro sul risultato.
- **Colonna username** sulla tabella utenti, distinta dall'email: e' il nome pubblico dell'utente (univoco su tutta la tabella, indipendentemente dal ruolo), mostrato come autore delle recensioni; l'email resta invece l'identificativo privato di accesso (unico per ruolo, mai esposto ad altri utenti).

- **Cucine come tabella a parte + associazione N:M** (cucine, ristoranti_cucine): un ristorante puo' avere piu' tipi di cucina (es. "Creative, Contemporary" nel dataset Michelin) e un tipo di cucina e' condiviso da piu' ristoranti; un campo testuale libero avrebbe introdotto ridondanza e reso inefficiente la ricerca per tipologia di cucina.
- **Vincolo UNIQUE(id_ristorante, id_cliente)** sulla tabella recensioni: riflette la regola "un cliente inserisce/modifica/cancella le proprie recensioni" (al singolare per ristorante), impedendo recensioni duplicate dello stesso cliente sullo stesso ristorante.
- **Risposta del ristorante come attributo della recensione** (risposta_testo, data_risposta) anziche' come tabella separata: e' una relazione 1:1 opzionale strettamente dipendente dalla recensione (al massimo una risposta per recensione, come da specifica).
- **Preferiti** come tabella con chiave primaria composta (id_cliente, id_ristorante): la coppia identifica univocamente la riga, senza bisogno di un id surrogato.
- **Vincoli applicativi non esprimibili con FK/CHECK dichiarativi** (es. "id_ristoratore deve riferire un utente con ruolo RISTORATORE") sono implementati con trigger PL/pgSQL, cosi' che l'integrita' sia garantita indipendentemente dal client che scrive (vedi funzioni fn_check_ruolo_ristoratore, fn_check_ruolo_cliente, fn_check_ruolo_cliente_preferiti in schema.sql).

5. Protocollo di comunicazione client/server

Il contratto dei servizi e' definito una sola volta nell'interfaccia theknife.common.ServiziTK, con la stessa nomenclatura delle funzionalita' elencate nel documento di specifica (cercaRistorante, visualizzaRistorante, visualizzaRecensioni, registrazione, aggiungiPreferito, rimuoviPreferito, visualizzaPreferiti, aggiungiRecensione, modificaRecensione, eliminaRecensione, aggiungiRistorante, visualizzaRiepilogo, rispostaRecensione, ecc.).

Ogni chiamata lato client (ServiziTKProxy) costruisce un oggetto Richiesta(Operazione, Object... parametri), lo invia sul socket e attende un oggetto Risposta. Lato server, GestoreClient smista la richiesta in base al valore dell'enum Operazione e, per le operazioni riservate a un ruolo, verifica che l'utente autenticato sulla connessione corrente coincida con l'id dichiarato nella richiesta e abbia il ruolo atteso, prima di delegare a ServiziTKImpl.

Nota di sicurezza: il controllo di autorizzazione lato server non si fida ciecamente dei parametri inviati dal client (che potrebbero, in teoria, essere manipolati da un client malevolo): l'id utente dichiarato viene sempre confrontato con l'utente effettivamente autenticato su quella connessione (verificato con login/registrazione).

6. Struttura del codice sorgente

Tutto il codice e' definito nel package theknife, con i seguenti sottopackage:

theknife.common	classi condivise da client e server
theknife.common.model	Utente (astratta), Cliente, Ristoratore, Ristorante, Recensione, Ruolo
theknife.common.protocol	Operazione, Richiesta, Risposta, CriteriRicerca
theknife.common.util	TheKnifeException, PasswordUtil (hash BCrypt)
theknife.server	ServerTK (main), GestoreClient, ConsoleUtil, ImportCSV
theknife.server.dao	ConfigurazioneDB, UtenteDAO, RistoranteDAO, RecensioneDAO, PreferitoDAO, UtenteCredenziali
theknife.server.service	ServiziTKImpl (Facade)

theknife.client	ClientTK (main JavaFX), LauncherClient, Navigatore, SessioneClient, EseguiAsync, Formattatori
theknife.client.rete	ConnessioneServer, ServiziTKProxy
theknife.client.controller	controller delle 11 schermate FXML

I due eseguibili richiesti dalla consegna sono prodotti dal medesimo modulo Maven tramite due esecuzioni del plugin maven-shade-plugin con Main-Class diverse: theknife.server.ServerTK per theknife-server.jar e theknife.client.LauncherClient per theknife-client.jar (vedi pom.xml).

7. Gestione della concorrenza

- **Lato server:** un thread per connessione client (GestoreClient). Ogni DAO apre una connessione JDBC dedicata per ciascuna operazione (try-with-resources): non essendoci alcuna connessione o struttura dati condivisa mutabile tra i thread, non e' necessaria sincronizzazione esplicita in Java; la concorrenza sull'accesso ai dati condivisi (stesso ristorante, stessa recensione) e' delegata al DBMS stesso, che la gestisce tramite le proprie transazioni e i vincoli definiti sullo schema (MVCC di PostgreSQL).
- **Lato client:** le chiamate di rete non sono mai eseguite sul thread grafico JavaFX (che deve restare libero per rimanere reattivo); EseguiAsync le esegue su un thread separato e riporta il risultato sul thread JavaFX con Platform.runLater.
- ConnessioneServer.invia() e' sincronizzato: il protocollo e' rigorosamente request/response su un'unica connessione, quindi due chiamate di rete avviate in parallelo dallo stesso client non devono interferire le rispettive richieste/risposte.

La correttezza in presenza di piu' client connessi contemporaneamente e' stata verificata con un test automatico che avvia 30 client concorrenti, ciascuno dei quali esegue 5 ricerche in sequenza sulla propria connessione: tutte le 150 richieste sono state completate con successo senza errori ne' interferenze tra le sessioni.

8. Controllo degli errori e gestione delle eccezioni

Si distinguono due categorie di errore:

- **Errori di business** (email gia' registrata, credenziali errate, recensione non trovata o non propria, operazione non autorizzata, stelle fuori range, ecc.): segnalati con l'eccezione *checked* TheKnifeException, con un messaggio pensato per essere mostrato direttamente all'utente nella UI.
- **Errori tecnici** (SQLException, problemi di rete): intercettati lato server, loggati con stack trace completo (java.util.logging) e tradotti in un messaggio generico prima di essere restituiti al client, per non esporre dettagli implementativi (nomi di tabelle, query) all'esterno.

Lato client, ogni operazione di rete e' racchiusa in un blocco try/catch (tramite EseguiAsync): in caso di errore, il messaggio viene mostrato in un'etichetta dedicata della schermata corrente, senza mai interrompere l'applicazione.

9. Import dei dati draft (michelin_my_maps.csv)

La classe `theknife.server.ImportCSV` importa il file fornito dal docente nella tabella ristoranti, come dati iniziali di popolamento (17.737 ristoranti importati con successo nei test). Poiché il formato del dataset Michelin non coincide con lo schema `TheKnife`, sono state necessarie alcune trasformazioni, documentate anche nella Javadoc della classe:

- **Nazione/Città':** ricavate dalla colonna `Location` ("Città', Nazione"); per le città-stato senza virgola (es. Singapore, Hong Kong) nazione e città' coincidono.
- **Fascia di prezzo:** la colonna `Price` del dataset non è un importo ma un simbolo di valuta locale ripetuto da 1 a 4 volte (es. "€€€"); non essendo disponibile un importo reale né un tasso di cambio storico per le valute non euro, il numero di simboli è stato tradotto in una fascia di prezzo medio convenzionale in euro (1 simbolo = 25€, 2 = 45€, 3 = 80€, 4 = 150€).
- **Delivery / prenotazione online:** non presenti nel dataset Michelin, impostati a "no" per i record importati (un ristorante reale li imposta correttamente creando il proprio ristorante con `aggiungiRistorante()`).
- **Proprietario:** ogni ristorante deve appartenere a un utente con ruolo `RISTORATORE` (vincolo `NOT NULL + trigger`); i ristoranti importati sono assegnati a un account "seed" dedicato (`import@theknife.local`), creato automaticamente al primo import.

10. Sicurezza

Le password non sono mai memorizzate né trasmesse in chiaro oltre il messaggio di login/registrazione stesso: sul database è salvato solo l'hash `BCrypt` (`PasswordUtil`, libreria `BCrypt`), funzione di hash intenzionalmente lenta e con salt integrato, pensata per le password (a differenza di un semplice `SHA-256`, rende impraticabili gli attacchi a forza bruta/dizionario). Gli oggetti `Utente` restituiti dal server al client non contengono mai l'hash della password.

In un contesto di produzione reale, oltre all'autenticazione, andrebbe cifrato anche il canale di trasporto (es. tramite `SSL/TLS`): per un progetto di laboratorio su rete fidata questo aspetto è stato ritenuto fuori scopo, ma è segnalato come possibile estensione futura.

11. Artefatti di progettazione

I diagrammi prodotti sono disponibili come file vettoriali `SVG` nella cartella `doc/` del repository (visualizzabili in qualunque browser):

- **ER_TheKnife.svg:** schema Entity-Relationship concettuale del database `dbTK`.
- **UML_Dominio_TheKnife.svg:** diagramma delle classi UML del modello di dominio (`Utente/Cliente/Ristoratore`, `Ristorante`, `Recensione`) e dell'interfaccia `ServiziTK`.
- **UML_Architettura_TheKnife.svg:** diagramma architetturale (package `client/common/server`, pattern `Facade/DAO/Proxy/MVC`, canale di comunicazione `client-server`).

La documentazione Javadoc completa del codice sorgente è generata con `mvn javadoc:javadoc` in `doc/javadoc/apidocs/index.html`.

12. Limitazioni note e possibili sviluppi futuri

- Le connessioni JDBC sono aperte e chiuse per singola operazione anziché tramite un vero connection pool (es. HikariCP): scelta che privilegia la semplicità del codice per un progetto di laboratorio; sotto carico molto elevato un pool ridurrebbe l'overhead di apertura connessione.
- Il canale di rete non è cifrato (nessun TLS): adeguato a un contesto di laboratorio, da rivedere per un utilizzo su rete non fidata.
- La ricerca per luogo confronta testualmente città/nazione (con corrispondenza parziale), senza calcolare una vera distanza geografica a partire dalle coordinate: un geocoding completo del testo libero digitato dall'utente richiederebbe un servizio esterno, ritenuto fuori scopo per il progetto.